

```

# =====
# CORRECTED PIPELINE 2.0
# STAGE 1 — RAW DATA AUDIT AND EXPLORATORY DATA ANALYSIS
# =====
import os
import json
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")
print("=" * 72)
print("CORRECTED PIPELINE 2.0")
print("STAGE 1 — RAW DATA AUDIT AND EXPLORATORY DATA ANALYSIS")
print("=" * 72)

# --- 1. PATHS -----
raw_data_path = (
    r"D:\Data_Workspace\Capstone_Project"
    r"\NLGN_PLC_Predictive_Analytic\data\01_raw"
    r"\NLNG_Dirty_Data.csv"
)
corrected_root = (
    r"D:\Data_Workspace\Capstone_Project"
    r"\NLGN_PLC_Predictive_Analytic\data"
    r"\04_corrected_pipeline"
)
stage1_dir = os.path.join(corrected_root, "stage_1_audit_eda")
os.makedirs(stage1_dir, exist_ok=True)
print(f"\nRaw dataset:\n{raw_data_path}")
print(f"\nCorrected pipeline output:\n{corrected_root}")

# --- 2. LOAD RAW DATA -----
raw_df = pd.read_csv(raw_data_path)
# Standardise headers only. Values are not modified.
raw_df.columns = raw_df.columns.str.strip().str.lower()
print("\n" + "=" * 72)
print("RAW DATASET")
print("=" * 72)
print(f"Rows: {len(raw_df):,}")
print(f"Columns: {len(raw_df.columns):,}")
mem_mb = raw_df.memory_usage(deep=True).sum() / 1024**2
print(f"Memory: {mem_mb:.2f} MB")
print("\nFirst 5 rows:")
print(raw_df.head())

# --- 3. SCHEMA AUDIT -----
schema_report = pd.DataFrame({
    "column": raw_df.columns,
    "dtype": [str(raw_df[c].dtype) for c in raw_df.columns],
    "missing_count": [raw_df[c].isna().sum()
                      for c in raw_df.columns],
    "missing_pct": [raw_df[c].isna().mean() * 100
                    for c in raw_df.columns],
    "unique_values": [raw_df[c].nunique(dropna=False)
                      for c in raw_df.columns],
})
schema_report = schema_report.sort_values("missing_pct",
                                          ascending=False)
schema_report.to_csv(os.path.join(stage1_dir,
                                  "01_schema_audit.csv"),
                    index=False)
print("\nSchema audit saved.")

# --- 4. DUPLICATE AUDIT -----
exact_duplicates = raw_df.duplicated().sum()
if {"equipment_id", "timestamp"}.issubset(raw_df.columns):
    equipment_timestamp_duplicates = raw_df.duplicated(
        subset=["equipment_id", "timestamp"],
        keep=False).sum()
else:
    equipment_timestamp_duplicates = np.nan
duplicate_report = pd.DataFrame({
    "audit_item": ["exact_duplicate_rows",
                  "equipment_timestamp_duplicate_rows"],
    "count": [exact_duplicates,
              equipment_timestamp_duplicates],
})
duplicate_report.to_csv(os.path.join(stage1_dir,
                                     "02_duplicate_audit.csv"),
                      index=False)
print("\nDuplicate audit:")
print(duplicate_report)

# --- 5. TIMESTAMP AUDIT -----
raw_df["timestamp"] = pd.to_datetime(raw_df["timestamp"],
                                     errors="coerce")
timestamp_report = pd.DataFrame({
    "metric": ["invalid_or_missing_timestamps",
              "minimum_timestamp", "maximum_timestamp",
              "unique_timestamps"],
    "value": [raw_df["timestamp"].isna().sum(),
              raw_df["timestamp"].min(),
              raw_df["timestamp"].max(),
              raw_df["timestamp"].nunique(),
              ],
})
timestamp_report.to_csv(os.path.join(stage1_dir,
                                     "03_timestamp_audit.csv"),
                      index=False)

print("\nTimestamp audit:")
print(timestamp_report)

# --- 6. EQUIPMENT AND TRAIN COVERAGE -----
if "equipment_id" in raw_df.columns:
    equipment_coverage = (
        raw_df["equipment_id"].value_counts()
        .rename_axis("equipment_id")
        .reset_index(name="record_count")
    )
    equipment_coverage.to_csv(os.path.join(stage1_dir,
                                           "04_equipment_coverage.csv"),
                            index=False)
if "train" in raw_df.columns:
    train_coverage = (
        raw_df["train"].value_counts().rename_axis("train")
        .reset_index(name="record_count")
    )
    train_coverage.to_csv(os.path.join(stage1_dir,
                                       "05_train_coverage.csv"),
                        index=False)
n_eq = (raw_df["equipment_id"].nunique()
        if "equipment_id" in raw_df.columns else "N/A")
n_tr = (raw_df["train"].nunique()
        if "train" in raw_df.columns else "N/A")
print(f"\nEquipment assets: {n_eq}")
print(f"Trains: {n_tr}")

# --- 7. MISSING VALUE AUDIT -----
missing_report = pd.DataFrame({
    "column": raw_df.columns,
    "missing_count": raw_df.isna().sum().values,
    "missing_percentage": raw_df.isna().mean().values * 100,
})
missing_report = missing_report.sort_values("missing_percentage",
                                           ascending=False)
missing_report.to_csv(os.path.join(stage1_dir,
                                   "06_missing_value_audit.csv"),
                    index=False)
print("\n" + "=" * 72)
print("MISSING VALUE AUDIT")
print("=" * 72)
print(missing_report[missing_report["missing_count"] > 0])

# --- 8. TARGET AUDIT — FAILURE WITHIN 24 HOURS ---
classification_target = "failure_within_24h"
if classification_target in raw_df.columns:
    failure_counts = (raw_df[classification_target]
                      .value_counts(dropna=False).sort_index())
    failure_distribution = pd.DataFrame({
        "count": failure_counts,
        "percentage": failure_counts / len(raw_df) * 100,
    })
    failure_distribution.to_csv(os.path.join(stage1_dir,
                                             "07_failure_24h_distribution.csv"))
    print("\n" + "=" * 72)
    print("FAILURE WITHIN 24 HOURS")
    print("=" * 72)
    print(failure_distribution)

# --- 9. RUL AUDIT -----
rul_target = "rul_days"
if rul_target in raw_df.columns:
    rul_audit = pd.DataFrame({
        "metric": ["missing", "negative", "mean", "median",
                  "minimum", "maximum"],
        "value": [raw_df[rul_target].isna().sum(),
                  (raw_df[rul_target] < 0).sum(),
                  raw_df[rul_target].mean(),
                  raw_df[rul_target].median(),
                  raw_df[rul_target].min(),
                  raw_df[rul_target].max(),
                  ],
    })
    rul_audit.to_csv(os.path.join(stage1_dir,
                                  "08_rul_audit.csv"),
                    index=False)
    print("\n" + "=" * 72)
    print("RUL AUDIT")
    print("=" * 72)
    print(rul_audit)

# --- 10. PHYSICAL VALIDITY AUDIT -----
physical_rules = {
    "load_factor": (0, 1), "wear_level": (0, 1),
    "lubrication_health_index": (0, 100),
    "oil_pressure": (0, np.inf),
    "oil_particles_ppm": (0, np.inf),
    "overall_vibration": (0, np.inf),
    "vibration": (0, np.inf), "rpm": (0, np.inf),
    "hours_since_maint": (0, np.inf),
    "cumulative_op_hours": (0, np.inf),
    "rul_days": (0, np.inf),
}
physical_results = []
for col, (lower, upper) in physical_rules.items():

```

```

    if col not in raw_df.columns:
        continue
    invalid_mask = ((raw_df[col] < lower)
                    | (raw_df[col] > upper)) &
    raw_df[col].notna()
    physical_results.append({
        "variable": col, "lower_bound": lower,
        "upper_bound": upper,
        "invalid_count": int(invalid_mask.sum()),
        "invalid_pct": float(invalid_mask.mean() * 100),
    })
physical_report = pd.DataFrame(physical_results)
physical_report.to_csv(os.path.join(stage1_dir,
                                    "09_physical_validity_audit.csv"),
                      index=False)

print("\n" + "=" * 72)
print("PHYSICAL VALIDITY AUDIT")
print("=" * 72)
print(physical_report)

# --- 11. OUTLIER AUDIT — IQR -----
numeric_columns =
    raw_df.select_dtypes(include=np.number).columns
outlier_results = []
for col in numeric_columns:
    series = raw_df[col].dropna()
    if len(series) == 0:
        continue
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    mask = (raw_df[col] < lower) | (raw_df[col] > upper)
    outlier_results.append({
        "variable": col, "lower_iqr": lower,
        "upper_iqr": upper,
        "outlier_count": int(mask.sum()),
        "outlier_pct": float(mask.mean() * 100),
    })
outlier_report = (pd.DataFrame(outlier_results)
                  .sort_values("outlier_pct",
                              ascending=False))
outlier_report.to_csv(os.path.join(stage1_dir,
                                    "10_outlier_audit_iqr.csv"),
                      index=False)

print("\n" + "=" * 72)
print("TOP OUTLIER VARIABLES")
print("=" * 72)
print(outlier_report.head(15))

# --- 12. LEAKAGE SCREENING -----
potential_leakage = ["failure_within_72h",
                    "failure_within_7d",
                    "rul_days", "rul_censored",
                    "failure_rate"]
treatments = []
for c in potential_leakage:
    if c in ["failure_within_72h", "failure_within_7d"]:
        treatments.append("EXCLUDE — future failure target")
    elif c in ["rul_days", "rul_censored"]:
        treatments.append("EXCLUDE — regression target")
    elif c == "failure_rate":
        treatments.append("EXCLUDE CONSERVATIVELY — verify
        provenance")
    else:
        treatments.append("Not present")
leakage_report = pd.DataFrame({
    "variable": potential_leakage,
    "present": [c in raw_df.columns for c in
    potential_leakage],
    "treatment": treatments,
})
leakage_report.to_csv(os.path.join(stage1_dir,
                                    "11_leakage_screening.csv"),
                      index=False)

print("\n" + "=" * 72)
print("LEAKAGE SCREENING")
print("=" * 72)
print(leakage_report)

# --- 13. DESCRIPTIVE STATISTICS -----
numeric_profile = (raw_df.select_dtypes(include=np.number)
                  .describe().T)
numeric_profile["skewness"] = (raw_df
                              .select_dtypes(include=np.number)
                              .skew())
numeric_profile.to_csv(os.path.join(stage1_dir,
                                    "12_raw_statistical_profile.csv"))

print("\n" + "=" * 72)
print("RAW NUMERICAL PROFILE")
print("=" * 72)
print(numeric_profile)

# --- 14. RUL DISTRIBUTION -----
if rul_target in raw_df.columns:
    plt.figure(figsize=(8, 5))
    raw_df[rul_target].dropna().plot(kind="hist", bins=50)
    plt.xlabel("RUL (Days)")
    plt.ylabel("Frequency")
    plt.title("Raw RUL Distribution")
    plt.tight_layout()
    plt.savefig(os.path.join(stage1_dir,
                              "eda_rul_distribution.png"),
                dpi=300)
    plt.show()
    plt.close()

# --- 15. FAILURE DISTRIBUTION -----
if classification_target in raw_df.columns:
    failure_counts = (raw_df[classification_target]
                     .value_counts().sort_index())
    plt.figure(figsize=(7, 5))
    failure_counts.plot(kind="bar")
    plt.xlabel("Failure Within 24 Hours")
    plt.ylabel("Number of Observations")
    plt.title("24-Hour Failure Class Distribution")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.savefig(os.path.join(stage1_dir,
                              "eda_failure_distribution.png"),
                dpi=300)
    plt.show()
    plt.close()

# --- 16. KEY SENSOR DISTRIBUTIONS -----
key_sensors = ["overall_vibration", "vibration",
               "oil_particles_ppm", "oil_pressure",
               "lubrication_health_index", "load_factor",
               "wear_level"]
available_sensors = [c for c in key_sensors if c in
                     raw_df.columns]
for col in available_sensors:
    plt.figure(figsize=(7, 5))
    raw_df[col].dropna().plot(kind="hist", bins=50)
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.title(f"Distribution of {col}")
    plt.tight_layout()
    plt.savefig(os.path.join(stage1_dir,
                              f"eda_{col}_distribution.png"),
                dpi=300)
    plt.show()
    plt.close()

# --- 17. SENSOR VALUES BY FAILURE STATUS -----
if classification_target in raw_df.columns:
    sensor_failure_summary = []
    for col in available_sensors:
        normal = raw_df.loc[raw_df[classification_target] ==
                             0,
                             col].dropna()
        failure = raw_df.loc[raw_df[classification_target] ==
                              1,
                              col].dropna()
        sensor_failure_summary.append({
            "sensor": col, "normal_n": len(normal),
            "failure_n": len(failure),
            "normal_median": normal.median(),
            "failure_median": failure.median(),
            "median_difference": failure.median() -
            normal.median(),
        })
    sensor_failure_report =
    pd.DataFrame(sensor_failure_summary)
    sensor_failure_report.to_csv(os.path.join(stage1_dir,
                                                "13_sensor_failure_comparison.csv"),
                                index=False)

    print("\n" + "=" * 72)
    print("SENSOR VALUES BY FAILURE STATUS")
    print("=" * 72)
    print(sensor_failure_report)

# --- 18. STAGE 1 SUMMARY -----
stage1_summary = {
    "rows": int(len(raw_df)),
    "columns": int(len(raw_df.columns)),
    "exact_duplicates": int(exact_duplicates),
    "equipment_timestamp_duplicates": (
        int(equipment_timestamp_duplicates)
        if not pd.isna(equipment_timestamp_duplicates) else
        None),
    "missing_value_columns": int((raw_df.isna().sum() >
    0).sum()),
    "equipment_assets": (int(raw_df["equipment_id"].nunique())
                        if "equipment_id" in raw_df.columns
                        else None),
    "trains": (int(raw_df["train"].nunique())
              if "train" in raw_df.columns else None),
}
with open(os.path.join(stage1_dir, "stage1_summary.json"),
          "w") as f:
    json.dump(stage1_summary, f, indent=4)
print("\n" + "=" * 72)
print("STAGE 1 COMPLETE")
print("=" * 72)
print(json.dumps(stage1_summary, indent=4))

```